

INFORME IMPLEMENTACION PROYECTO FINAL JUEGO EN C++

ELABORADO POR: JUAN ESTEBAN SOLARTE LOPEZ

CC.1081052976

CORREO INSTITUCIONAL: juan.solarte1@udea.edu.co

CURSO: INFORMATICA II

UNIVERSIDAD DE ANTIOQUIA

SEMESTRE ACADEMICO 2026-1

MEDELLIN-ANTIOQUIA

06/06/2026

TITULO: PÉRTIGA EN EL REINO MINIATURA

Repositorio: <https://github.com/Jsolarte8/Proyecto-Final-Info-2-.git>

1. INTRODUCCION

El proyecto consiste en el desarrollo de un videojuego sencillo implementado en C++ usando la biblioteca Qt para la interfaz gráfica. El juego se llama **Pértiga en el Reino Miniatura** y propone una experiencia de aventura deportiva en un universo especial, donde el personaje principal debe superar obstáculos, recolectar elementos y avanzar entre niveles usando habilidades relacionadas con el salto, la movilidad y el uso de una pértiga.

El desarrollo busca aplicar los conceptos principales trabajados en el curso: programación orientada a objetos, memoria dinámica, uso de interfaces gráficas, manejo de eventos, colisiones, temporizadores, contenedores y modelos físicos parametrizables. Además, el proyecto incorpora un agente inteligente sencillo representado por la **Araña Guardiana**, que percibe el comportamiento del jugador, razona sobre las zonas más visitadas, actúa colocando hilos y aprende a partir del historial de movimiento.

El público objetivo del videojuego son usuarios que buscan una experiencia corta, visualmente clara y con dificultad progresiva. El juego se organiza en niveles con dinámicas diferenciadas: un primer nivel de vista lateral con salto, plataformas, obstáculos y temporizador; y un segundo nivel de desplazamiento cenital donde aparece el comportamiento inteligente de la araña.

2. PLANTEAMIENTO DEL PROBLEMA

El problema planteado consiste en diseñar e implementar un videojuego que combine una temática deportiva con un universo especial, cumpliendo con restricciones académicas específicas: uso de C++, Qt, programación orientada a objetos, memoria dinámica, físicas propias y un agente inteligente sencillo.

La necesidad que cubre el software es demostrar, mediante una aplicación funcional e interactiva, que los conceptos de arquitectura de software, diseño de clases, herencia, composición, polimorfismo, contenedores y control de eventos pueden integrarse en un sistema gráfico. El juego no se limita a mover elementos en pantalla, sino que incorpora reglas, estados, dificultad, interacción entre objetos, control de tiempo, sonidos y toma de decisiones por parte de un enemigo.

El proyecto se justifica porque permite llevar los conceptos teóricos del curso a una solución concreta. Cada clase cumple una responsabilidad específica y el sistema se separa en dos capas principales: la capa lógica, encargada de las reglas del juego, entidades, físicas y niveles; y la capa GUI, encargada de la visualización, sonidos, entrada del teclado y navegación entre pantallas.

3. DEFINICIÓN GENERAL Y OBJETIVOS

3.1 Objetivo general

Desarrollar un videojuego en C++ y Qt que implemente programación orientada a objetos, memoria dinámica, modelos físicos, dificultad configurable, interfaz gráfica y un agente inteligente simple.

3.2 Objetivos específicos

- Implementar una arquitectura basada en clases, separando la lógica del juego y la interfaz gráfica.
- Usar herencia propia en la jerarquía de entidades y personajes.
- Implementar memoria dinámica mediante creación y destrucción de niveles, jugador, obstáculos, recolectables, plataformas y enemigos.
- Incluir al menos tres modelos físicos: movimiento parabólico, comportamiento elástico de la pértiga y fricción.
- Incorporar dificultad configurable que modifique parámetros del juego.
- Implementar un agente inteligente con percepción, razonamiento, acción y aprendizaje.
- Crear una interfaz gráfica en Qt con menús, selección de personaje, selección de dificultad, escena de juego y batalla final.
- Integrar sonido de ambiente y sonidos de eventos.
- Gestionar el proyecto en un repositorio con commits que evidencien el avance.

4. ESPECIFICACIÓN DE REQUERIMIENTOS

4.1 Requerimientos funcionales

1. El sistema debe permitir iniciar el videojuego desde una ventana principal.
2. El sistema debe mostrar una pantalla introductoria.
3. El sistema debe permitir seleccionar personaje.
4. El sistema debe permitir seleccionar dificultad.
5. El sistema debe ejecutar el nivel uno con vista lateral, plataformas, obstáculos, recolectables y temporizador.
6. El sistema debe permitir mover al jugador, saltar, cargar la pértiga y usarla.
7. El sistema debe aplicar movimiento parabólico durante los saltos.
8. El sistema debe aplicar una física elástica asociada a la pértiga.
9. El sistema debe aplicar fricción cuando el jugador entra en zonas como charcos o hilos.
10. El sistema debe detectar colisiones entre jugador, plataformas, obstáculos y recolectables.
11. El sistema debe avanzar al nivel dos cuando se completa el nivel uno.
12. El sistema debe ejecutar el nivel dos con vista cenital y presencia de la Araña Guardiana.
13. El sistema debe permitir recolectar semillas para abrir la salida.
14. El sistema debe hacer que la Araña Guardiana perciba la zona del jugador, aprenda rutas, tome decisiones y coloque hilos.
15. El sistema debe mostrar una batalla final.
16. El sistema debe mostrar estados de victoria o derrota.
17. El sistema debe reproducir sonido de ambiente y sonidos de eventos como recolección o daño.

4.2 Requerimientos no funcionales

- El videojuego debe ser comprensible y usable para el usuario final.
- La interfaz debe ser visualmente clara.
- El código debe estar organizado en clases.
- El proyecto debe compilarse en Qt usando QMake.
- La dificultad debe modificar el comportamiento del juego sin limitarse solamente a reducir tiempo o aumentar cantidades.
- El sistema debe manejar memoria dinámica de forma responsable.
- El código debe permitir justificar decisiones de diseño durante la sustentación.

4.3 Restricciones

- El desarrollo se realiza en C++.
- La interfaz gráfica se implementa con Qt.
- Se usa programación orientada a objetos.
- Debe existir al menos una herencia propia no dependiente de clases Qt.
- Se deben incluir modelos físicos parametrizables.
- El diagrama de clases debe incluir capa lógica y capa GUI.

5. METODOLOGÍA Y PLANIFICACIÓN

La metodología aplicada fue incremental. Primero se definió la idea general del videojuego, sus niveles y mecánicas principales. Luego se diseñó la arquitectura mediante clases separadas para entidades, niveles, físicas, lógica del juego y pantallas de interfaz. Posteriormente se implementó la capa lógica, se conectó con la interfaz Qt y se añadieron elementos complementarios como sonidos, dificultad, batalla final y agente inteligente.

El desarrollo se puede dividir en las siguientes fases:

1. Contextualización: definición del universo, deporte base, dinámica general y niveles.
2. Diseño: identificación de clases, relaciones, contenedores, herencia y modelos físicos.
3. Implementación de lógica: creación de entidades, jugador, obstáculos, recolectables, niveles y físicas.
4. Implementación de GUI: creación de ventanas, pantallas, escenas gráficas, sonidos y eventos de teclado.
5. Integración: conexión entre la clase Juego y la pantalla GameScreen.
6. Pruebas: validación de movimiento, colisiones, avance de niveles, dificultad y comportamiento de la araña.
7. Entrega final: documentación, diagramas, video, trailer, código fuente y ejecutable.

6. DISEÑO Y ARQUITECTURA DEL SISTEMA

La arquitectura se organiza en dos capas principales:

6.1 Capa lógica

La capa lógica contiene las clases que representan las reglas internas del videojuego. Sus clases principales son:

- Juego: controlador general de la partida. Administra el nivel actual, puntaje, vidas, dificultad y cambio entre niveles.
- Nivel: clase abstracta base para los niveles. Define operaciones comunes como cargar, actualizar, verificar colisiones y mover al jugador.
- NivelUno: nivel lateral con plataformas, temporizador, saltos, pértiga, obstáculos y meta.
- NivelDos: nivel cenital con recolectables, hilos, salida y agente inteligente.
- Entidad: clase base para objetos con posición, tamaño, estado activo y rectángulo de colisión.
- Personaje: clase derivada de Entidad que agrega velocidad, estado y vidas.
- Jugador: personaje controlado por el usuario, con puntaje, néctar, semillas, energía de pértiga y estado de salto.
- Enemigo: personaje no jugable con agresividad, rango de visión y daño.
- AraniaGuardiana: enemigo inteligente que percibe, razona, actúa y aprende.
- Obstaculo, Recolectable y Plataforma: entidades del entorno.
- Fisica, FisicaParabolica, FisicaElastica y FisicaFriccion: clases encargadas de los modelos físicos.
- Temporizador: administra el tiempo disponible en el nivel uno.
- SistemaColisiones: centraliza funciones estáticas de colisión.

6.2 Capa de interfaz gráfica

La capa GUI se implementa con Qt. Sus clases principales son:

- MainWindow: ventana principal y controlador de navegación entre pantallas.
- IntroScreen: pantalla inicial con animación.
- MenuScreen: menú principal.
- CharacterSelectScreen: selección de personaje.
- DifficultyScreen: selección de dificultad.
- GameScreen: pantalla donde se renderizan los niveles, se procesan teclas, se actualiza el HUD y se reproducen sonidos.
- FinalBattleScreen: pantalla de batalla final contra la araña.

La conexión entre capas ocurre principalmente mediante GameScreen, que contiene un puntero a Juego. La GUI recibe eventos del teclado y los transforma en llamadas a la lógica, por ejemplo: mover jugador, saltar, cargar pértiga o usar

impulso. Luego la GUI reconstruye o actualiza la escena gráfica usando la información del nivel actual.

7. HERENCIA PROPIA

El proyecto usa herencia propia en dos jerarquías principales.

7.1 Jerarquía de entidades

Entidad es la clase base. De ella heredan Personaje, Obstaculo, Recolectable y Plataforma. Luego Personaje se especializa en Jugador y Enemigo, y finalmente AraniaGuardiana hereda de Enemigo.

Esta jerarquía permite reutilizar atributos comunes como posición, tamaño, estado activo y rectángulo de colisión. También permite especializar comportamientos según el tipo de objeto.

7.2 Jerarquía de físicas

Fisica es una clase abstracta con el método puro calcular(float deltaTiempo). De ella heredan FisicaParabolica, FisicaElastica y FisicaFriccion. Esta estructura permite representar diferentes modelos físicos bajo una interfaz común.

7.3 Jerarquía de niveles

Nivel es una clase abstracta. NivelUno y NivelDos heredan de ella y redefinen métodos como cargarNivel, actualizar, verificarColisiones y moverJugador. Esto permite que la clase Juego trabaje con un puntero Nivel* sin depender directamente de la implementación específica de cada nivel.

8. MODELOS FÍSICOS IMPLEMENTADOS

8.1 Movimiento parabólico

El movimiento parabólico se implementa en FisicaParabolica. Se usa principalmente para el salto del jugador en el nivel uno. El modelo considera posición, velocidad horizontal, velocidad vertical y gravedad. La gravedad modifica la velocidad vertical en cada actualización, generando una trayectoria curva.

Este modelo se usa cuando el jugador salta o cuando usa la pértiga para impulsarse.

8.2 Física elástica

La física elástica se implementa en FisicaElastica. Representa la carga y liberación de energía de la pértiga. El jugador puede cargar la pértiga y luego liberar esa energía para producir un impulso horizontal y vertical.

El modelo usa una constante elástica, amortiguación, carga y fuerza resultante. Esto permite que la acción de usar la pértiga sea más expresiva que un simple salto normal.

8.3 Física de fricción

La fricción se implementa en FisicaFriccion. Se aplica cuando el jugador entra en zonas que reducen el movimiento, como charcos en el nivel uno o hilos en el nivel dos. La fricción modifica la velocidad resultante según un coeficiente que cambia con la dificultad.

Este modelo permite que el entorno afecte la jugabilidad de forma visible.

9. DIFICULTAD

La dificultad se configura antes de iniciar la partida. El valor de dificultad modifica varios parámetros del juego:

- Daño de obstáculos.
- Coeficientes de fricción.
- Cantidad de obstáculos móviles.
- Tiempo disponible en el nivel uno.
- Agresividad de la Araña Guardiana.
- Daño producido por la araña.
- Frecuencia de aprendizaje de la araña.
- Cantidad máxima de hilos dinámicos.

De esta manera, la dificultad no se limita solamente a reducir el tiempo o aumentar objetos. También modifica el comportamiento del agente inteligente, la severidad de las penalizaciones físicas y la agresividad de los enemigos.

10. AGENTE INTELIGENTE: ARAÑA GUARDIANA

El agente inteligente del proyecto es la clase AraniaGuardiana, que hereda de Enemigo. Su arquitectura se divide en cuatro componentes:

10.1 Percepción

La araña percibe la posición del jugador y la convierte en una zona del mapa. Esta zona se representa como una cadena del tipo zona_x_y, calculada a partir de las coordenadas del jugador.

10.2 Razonamiento

La araña analiza su memoria de rutas para identificar las zonas más visitadas por el jugador. Si una zona ha sido visitada varias veces, la araña decide colocar un hilo en esa ubicación para bloquear o dificultar el paso.

10.3 Acción

La araña puede realizar acciones como perseguir al jugador o solicitar la colocación de un hilo dinámico. En el nivel dos, si la acción generada empieza con colocar_hilo, el nivel crea un nuevo obstáculo rectangular en la zona correspondiente.

10.4 Aprendizaje

El aprendizaje se realiza mediante un `std::map<std::string, int>` llamado memoriaRutas. Cada vez que el jugador pasa por una zona, la araña incrementa el contador de esa zona. Con esta información, el agente mejora sus decisiones y cambia el entorno durante la partida.

Este agente cumple con las etapas de percepción, razonamiento, acción y aprendizaje solicitadas para el componente investigativo.

11. CONTENEDORES UTILIZADOS

El proyecto utiliza contenedores estándar y de Qt:

11.1 std::vector

Se usa para almacenar colecciones de obstáculos, recolectables y plataformas dentro de la clase Nivel. También se usa en NivelDos para guardar los hilos dinámicos como rectángulos QRectF.

Justificación: std::vector permite recorrer elementos de forma eficiente y facilita agregar entidades durante la carga de niveles.

11.2 std::map

Se usa en AraniaGuardiana para representar la memoria de rutas. La clave es el nombre de la zona y el valor es la cantidad de veces que el jugador ha pasado por ella.

Justificación: std::map permite asociar una zona con un contador, lo cual es adecuado para el aprendizaje del agente.

11.3 std::queue

Se usa en AraniaGuardiana para organizar las acciones que el agente debe ejecutar, como perseguir o colocar un hilo.

Justificación: std::queue representa una cola de acciones en orden de llegada, lo cual facilita modelar el comportamiento del agente.

11.4 QVector

Se usa en la pantalla de selección de personajes para almacenar los personajes disponibles.

Justificación: QVector se integra bien con Qt y es útil para manejar datos dentro de la interfaz.

11.5 QSet

Se usa en las pantallas de juego para almacenar las teclas presionadas.

Justificación: QSet permite consultar rápidamente si una tecla está activa y evita duplicados.

12. MEMORIA DINÁMICA

El proyecto usa memoria dinámica mediante punteros. La clase Juego mantiene un puntero Nivel* nivelActual, que puede apuntar a un objeto NivelUno o NivelDos. Cuando se cambia o reinicia el nivel, se elimina el nivel anterior con delete y se crea uno nuevo con new.

La clase Nivel también administra dinámicamente objetos como jugador, obstáculos, recolectables y plataformas. Estos elementos se almacenan en contenedores de punteros. Para evitar fugas de memoria, se utiliza un proceso de limpieza antes de cargar un nuevo nivel o destruir el nivel.

En el nivel dos también se crea dinámicamente la AraniaGuardiana, que se elimina en el destructor correspondiente.

Este uso de memoria dinámica permite construir niveles con diferentes cantidades de entidades y liberar recursos cuando dejan de ser necesarios.

13. USO DE EXCEPCIONES

El proyecto incluye manejo de excepciones en la configuración de dificultad. En la clase Juego, el método setDifficulty(int difficulty) valida que la dificultad esté dentro del rango permitido. Si el valor no es válido, lanza una excepción de tipo std::invalid_argument.

Esto evita que el juego se ejecute con una configuración incorrecta y muestra una práctica de validación defensiva.

Ejemplo conceptual:

```
void Juego::setDifficulty(int difficulty) {
    if (difficulty < 0 || difficulty > 2) {
        throw std::invalid_argument("La dificultad debe estar entre 0 y 2.");
    }
    dificultad = difficulty;
}
```

La excepción es pertinente porque la dificultad afecta múltiples parámetros del sistema, como daño, tiempo, fricción y comportamiento del agente.

14. USO DE SEÑALES Y CONEXIONES EN QT

La interfaz usa el mecanismo de señales y slots de Qt para comunicar eventos entre pantallas y la ventana principal. Por ejemplo:

- IntroScreen emite introFinished() cuando termina la animación inicial.
- MenuScreen emite señales cuando el usuario selecciona iniciar juego, personajes, dificultad o salir.
- CharacterSelectScreen emite characterSelected(Character) cuando el usuario escoge un personaje.
- DifficultyScreen emite difficultySelected(int) cuando el usuario selecciona dificultad.
- GameScreen emite levelTwoFinished() o gameLost() según el resultado del juego.
- FinalBattleScreen emite finalBattleWon() o finalBattleLost().

La sintaxis general de conexión en Qt sigue la forma:

```
connect(objetoEmisor, &ClaseEmisora::senal,  
        objetoReceptor, &ClaseReceptora::slotOMetodo);
```

Este mecanismo permite separar las pantallas y evitar que una clase controle directamente todos los detalles de otra. La navegación se concentra en MainWindow.

15. DESARROLLO E IMPLEMENTACIÓN

La implementación se divide en módulos:

15.1 Control principal

La clase Juego administra la partida. Al iniciar, crea un objeto NivelUno. Cuando el nivel se completa, cambia a NivelDos. También almacena puntaje, vidas, dificultad y personaje seleccionado.

15.2 Nivel uno

El primer nivel se llama La Pista de las Hojas Altas. Tiene vista lateral, plataformas, obstáculos, recolectables y meta. El jugador debe avanzar usando saltos y la pértiga. El temporizador limita el tiempo disponible.

En este nivel se usan tres físicas principales: parábola para el salto, elasticidad para la pértiga y fricción para los charcos.

15.3 Nivel dos

El segundo nivel se llama La Red de la Araña Guardian. Tiene vista cenital. El jugador debe recolectar semillas para abrir la salida. En este nivel aparece la Araña Guardian, que aprende rutas y coloca hilos dinámicos.

La dificultad modifica la cantidad de hilos, el daño, el tiempo de reacción y la agresividad del agente.

15.4 Batalla final

Después de completar los niveles principales, se presenta una batalla final contra la araña. Esta pantalla tiene barras de vida, ataques, esquivar y temporizadores propios para controlar la actualización y los ataques enemigos.

16. PROCEDIMIENTOS DE PRUEBA

Se realizaron pruebas funcionales sobre los componentes principales del videojuego.

16.1 Pruebas de movimiento

Se verificó que el jugador pudiera moverse horizontalmente en el nivel uno y en varias direcciones en el nivel dos. También se validó que las teclas presionadas fueran registradas correctamente y que el estado del jugador cambiara entre reposo, caminar, saltar o cargar.

16.2 Pruebas de salto y pértiga

Se comprobó que el salto aplicara movimiento parabólico y que la pértiga produjera un impulso dependiente de la carga acumulada.

16.3 Pruebas de colisión

Se verificaron colisiones con plataformas, obstáculos, recolectables, hilos y meta. También se comprobó que los recolectables modificaran el puntaje, néctar o semillas del jugador.

16.4 Pruebas de dificultad

Se probó que al cambiar la dificultad se modificaran parámetros como tiempo, daño, fricción, cantidad de obstáculos, agresividad de la araña y máximo de hilos dinámicos.

16.5 Pruebas del agente inteligente

Se observó que la Araña Guardiana registrara las zonas visitadas por el jugador, seleccionara zonas repetidas y solicitara la colocación de hilos en dichas zonas.

16.6 Pruebas de interfaz

Se verificó la navegación entre introducción, menú, selección de personaje, selección de dificultad, juego, batalla final, victoria y derrota.

17. GUÍA DE INSTALACIÓN Y USO

17.1 Requisitos técnicos

- Qt Creator instalado.
- Qt con módulos core, gui, widgets y multimedia.
- Compilador compatible con C++17.
- Sistema operativo compatible con Qt.

17.2 Instalación

1. Descargar o clonar el repositorio del proyecto.
2. Abrir Qt Creator.
3. Seleccionar el archivo Info2ProyectoFinal.pro.
4. Configurar el kit de compilación.
5. Compilar el proyecto.
6. Ejecutar desde Qt Creator o desde el ejecutable generado.

17.3 Controles generales

- Teclas de dirección o WASD: mover personaje.
- Espacio: cargar o usar pértiga según el contexto.
- Teclas de acción definidas en la pantalla de juego: ataque, esquivar o habilidad especial durante la batalla final.

17.4 Objetivo del juego

En el nivel uno, el jugador debe avanzar hasta la meta antes de que se agote el tiempo. En el nivel dos, debe recolectar semillas, evitar los hilos de la araña y abrir la salida. Finalmente, debe superar la batalla contra la Araña Guardiana.

18. RESULTADOS Y DISCUSIÓN

El resultado final es un videojuego funcional con dos niveles principales, selección de personaje, dificultad configurable, interfaz gráfica, sonidos, entidades dinámicas, modelos físicos y un agente inteligente.

Uno de los aspectos más importantes del desarrollo fue la separación entre lógica e interfaz. La lógica se concentra en clases como Juego, Nivel, Jugador, AraniaGuardiana y las físicas. La interfaz se concentra en clases Qt como MainWindow, GameScreen y FinalBattleScreen.

Entre las dificultades enfrentadas se encuentran la coordinación entre eventos de teclado y actualización de la escena, el manejo de memoria dinámica, la detección de colisiones y la integración del comportamiento inteligente de la araña sin hacer que el juego fuera injusto.

Como mejora futura se podrían agregar más niveles, mejores animaciones, guardado de progreso, más enemigos, efectos visuales adicionales y una inteligencia más compleja para la araña.

19. CAMBIOS ENTRE LOS MOMENTOS ANTERIORES Y LA VERSIÓN FINAL

Durante los momentos iniciales se planteó la idea general del videojuego, los niveles y las físicas. En la versión final se realizaron ajustes para hacer el juego más implementable y coherente con el tiempo disponible.

Los principales cambios fueron:

- Se consolidó la temática del juego alrededor de la pértiga y el Reino Miniatura.
- Se organizó el sistema en dos capas: lógica y GUI.
- Se agregó una pantalla de selección de personaje.
- Se agregó selección de dificultad.
- Se implementó el primer nivel con salto, plataformas, recolectables, obstáculos y temporizador.
- Se implementó el segundo nivel con desplazamiento cenital, semillas, hilos y agente inteligente.
- Se añadió una batalla final para cerrar la experiencia.
- Se ajustaron los modelos físicos para que fueran evidentes durante el juego.
- Se incorporaron sonidos de ambiente y de eventos.
- Se refinó el agente inteligente para que almacenara zonas visitadas y actuara en consecuencia.

Estos cambios permitieron pasar de una propuesta conceptual a una implementación funcional.

20. CONCLUSIONES

El proyecto permitió aplicar los conceptos de programación orientada a objetos en un caso práctico. La construcción del videojuego exigió diseñar clases con responsabilidades claras, usar herencia, composición, memoria dinámica, contenedores y eventos de interfaz.

La separación entre la capa lógica y la capa gráfica facilitó la organización del código. La clase Juego actúa como controlador de la partida, mientras que GameScreen se encarga de conectar la lógica con la visualización en Qt.

Los modelos físicos implementados aportan variedad a la experiencia: el movimiento parabólico da naturalidad al salto, la física elástica permite la mecánica de la pértiga y la fricción modifica la movilidad en zonas específicas.

El agente inteligente de la Araña Guardiana representa un componente investigativo importante, ya que no solo persigue al jugador, sino que también registra información previa y adapta el entorno mediante hilos dinámicos.

En general, el proyecto cumplió con el objetivo de integrar POO, memoria dinámica, GUI, físicas, dificultad y agente inteligente dentro de un videojuego sencillo pero funcional.

21. REFERENCIAS

- Documentación oficial de Qt.
- Material del curso de Informática II.
- Repositorio del proyecto: <https://github.com/Jsolarte8/Proyecto-Final-Info-2-.git>
- Recursos gráficos y sonoros utilizados en el proyecto.

22. ANEXOS

Anexo A. Diagrama de clases completo

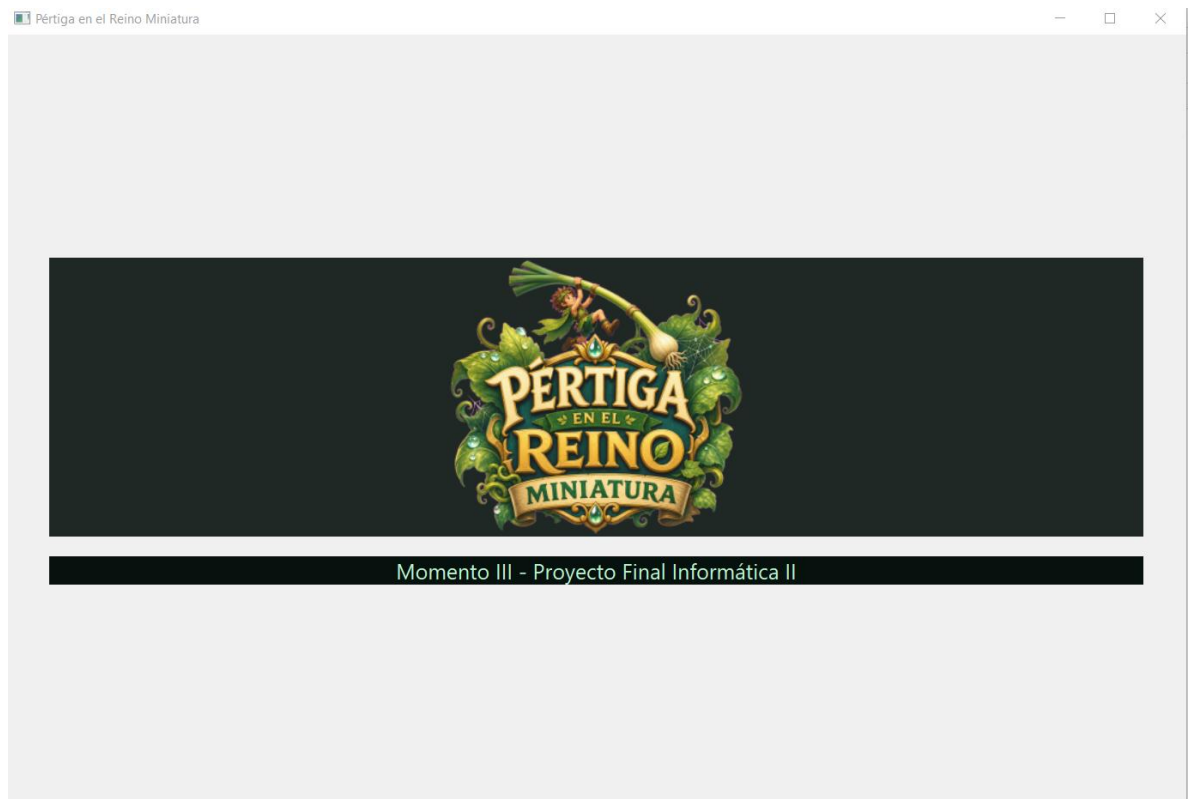
Diagrama de clases completo se encuentra anexo en la carpeta de entregables definitivos.

Anexo B. Diagrama de clases resumen

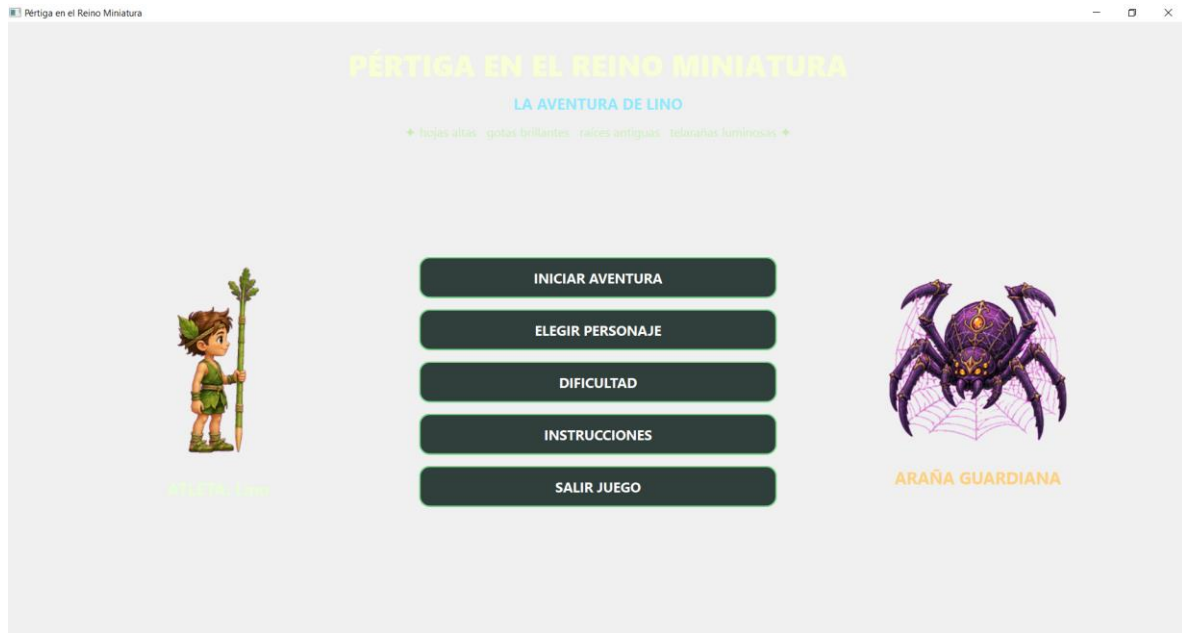
Diagrama de clases resumen se encuentra anexo en la carpeta de entregables definitivos.

Anexo C. Capturas del videojuego

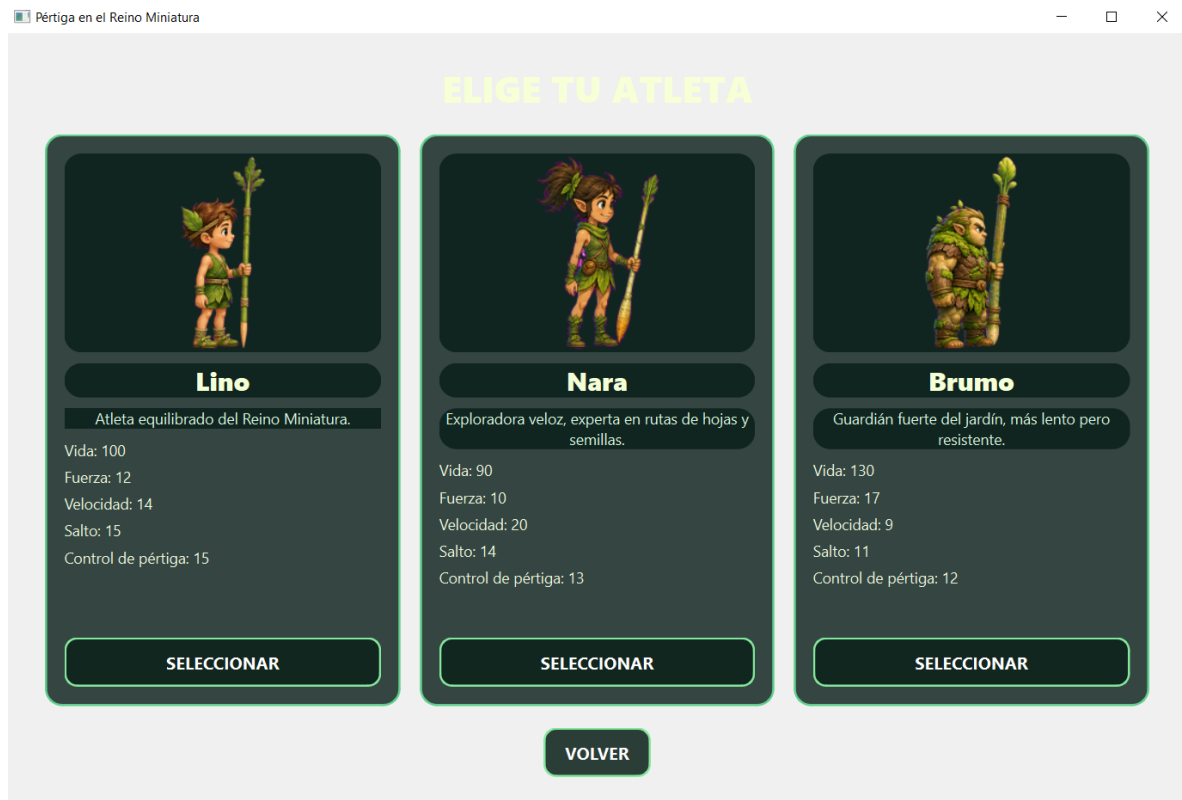
- Pantalla de inicio.



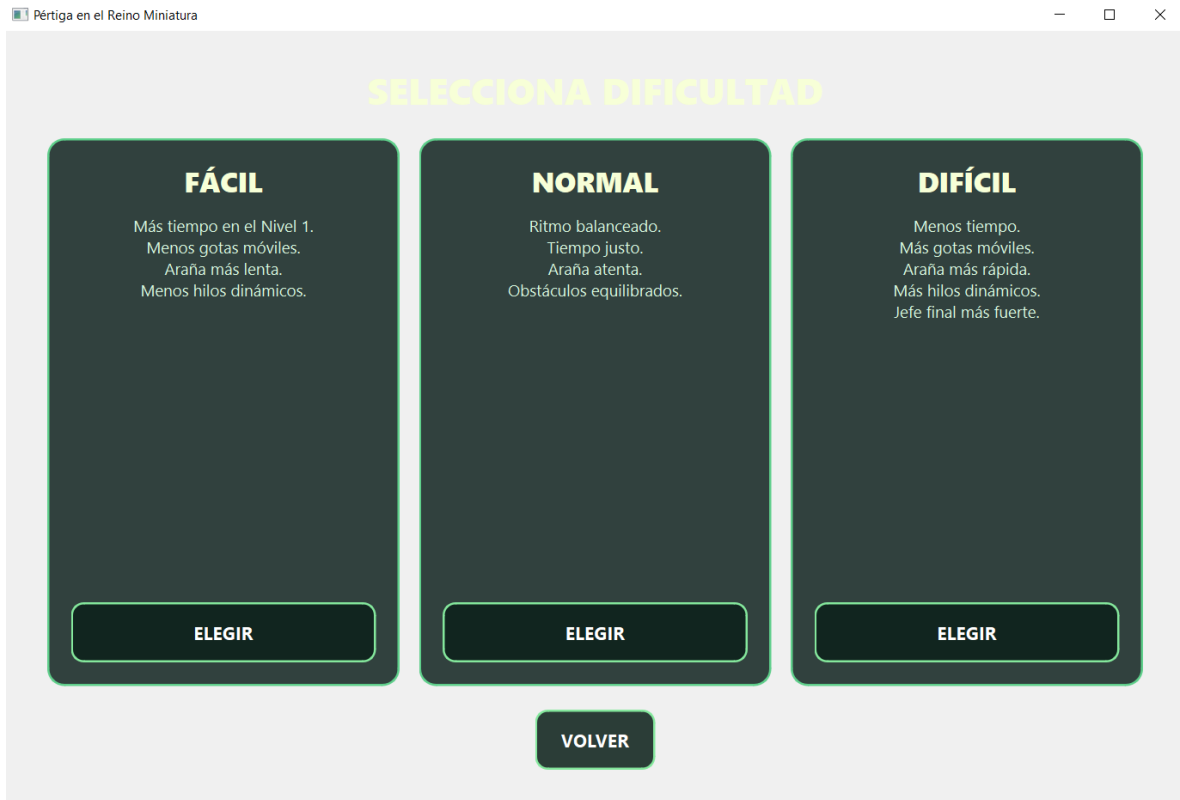
- Menú principal.



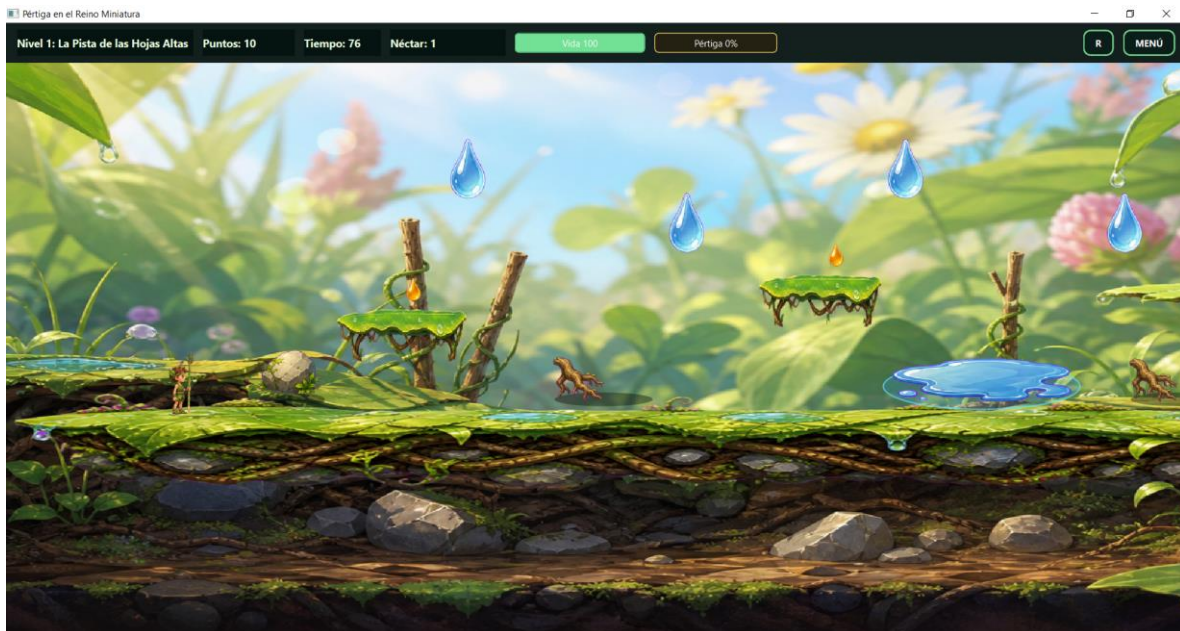
- Selección de personaje.



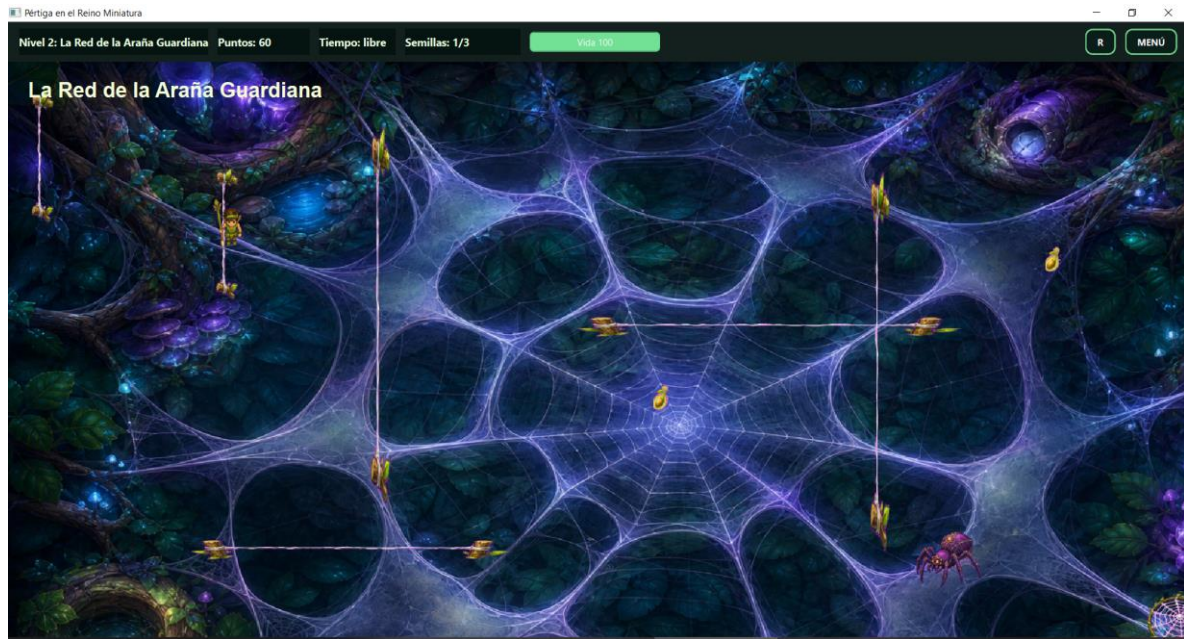
- Selección de dificultad.



- Nivel uno.



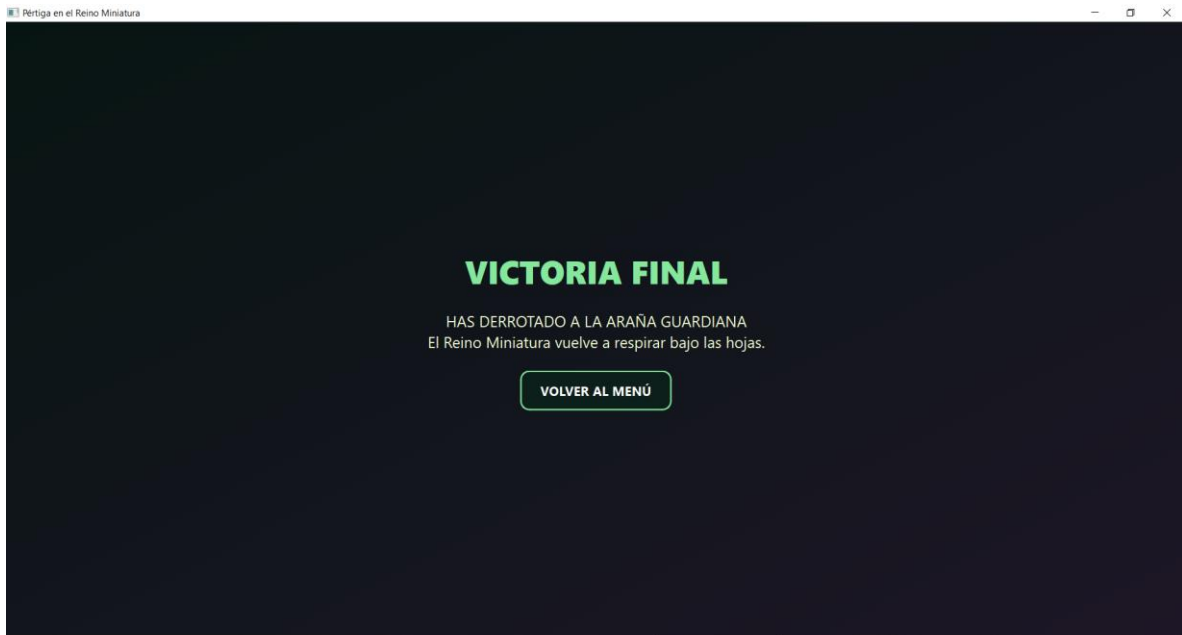
- Nivel dos.



- Batalla final.



- Pantalla de victoria o derrota.



Anexo D. Enlaces de video

Trailer: <https://www.youtube.com/watch?v=hKngEMDPWVI>